

Autonomous AI-Training for Robot-Assisted Sorting for Efficient Recycling (Team 3)
Austin Whitesky, Jin Kyong Park, Joon Yuan Chong, Lyon Kee
SENIOR SOFTWARE ENGINEERING PROJECT
(CS 462) 1/12/2024

Individual tasks

Unfortunately, no progress has been made by any individual over the past seven days as scheduling a meeting with the project partner has been unsuccessful. Consequently, no tasks have been completed or planned for the next progress report.

Meeting minutes

Meeting minutes with the team, project partner, or TA are not available due to the inability to schedule a meeting.

Open questions, concerns, or blockers

There are currently no open questions, concerns, or blockers to address.

Summary

1. What happened last week?

Nothing happened last week as it was still a break.

2. What happened this week?

This week we gathered our groups availability for the meetings. Besides that, we did not get to meet with our project partner as there was no reply to our email asking if they are available for a meeting. Our meeting with the TA also did not happen.

3. What happens next week?

We are planning a meeting on Sunday with our project partner and the team lead.

Issues completed this week

N/A, there are no issues set forth as of 1/12/2023.

We were able to figure out that we will be using the HPC provided by OSU for our cloud.

New Issues this week

N/A, there are no new issues for this week.

Backlog progress this week

N/A, there are no tasks/issues set forth as of 1/12/2023.

Major changes in assets this week

1. Product requirements document

No changes have been made. Still requires more information from collaborators.

2. Software development process

No changes have been made. Still requires more information from collaborators.

3. Software Design and Architecture

No changes have been made. Still requires more information from collaborators.

4. Product design, user persona, user research, etc.

No changes have been made. Still requires more information from collaborators.

Product Requirements Document (PRD)

Problem Description

Scope

Use Cases

Purpose and Vision (Background)

Stakeholders

Preliminary Context

Assumptions

Constraints

Dependencies

Market Assessment and Competition Analysis

Target Demographics (User Persona)

Requirements

User Stories and Features (Functional Requirements)

Non-Functional Requirements

Data Requirements

Integration Requirements

User Interaction and Design

Milestones and Timeline

Goals and Success Metrics

Open Questions

Out of Scope

Problem Description

There is one problem crucial to a recycling plant and an opportunity that we could capitalize on to enrich corporations with the data that they need to lead the market. The recycling sector pays very little, and there are very few workers who desire a job in that sector. Therefore it is better to allow a robot to work in this sector as the cost of electricity is significantly lower than human workers, especially in unhygienic and undesirable environments. The collection of data is also tedious for human workers and often results in mistakes, however, when we are using a robot, we are able to minimize the mistakes based on our model's accuracy and generate data that we can estimate the error rate. The data collection mentioned above pertains to the type of products that go in a recycling plant as this information is crucial to assist companies in identifying the consumption of their products as everything that is consumed becomes waste but not everything that a company sells will end up in the consumer's hands. It is the combination of societal problems and the lack of information that drives this project.

Scope

My team is focused on the automation of the problem where we build the model with YOLO and deploy it on the cloud. My team requires inputs from another team for 3-D object modeling in order to be used to identify recycling products. With the 3-D model, we are able to identify a normal can as well as a can that is squashed as generated from the 3-D model. The training is performed through Azure ML-OPS pipeline with the “You Only Look Once” classification technique. The trained model is then deployed onto an AWS cloud with Kubernetes. We are limited by our customers, for example, if Coca-Cola would like to pay for us to identify the amount of Coca-Cola cans that go through the facility, we would generate a 3D model to identify their product. Thus limiting us to only identifying stakeholders’ products and not more. The upper limit of our model is to identify slightly deformed cans due to the limited time and expertise that we have.

Use Cases

The first step is to upload the visual data collected from the camera onto the cloud. We then want to be able to identify the type of materials that pass through the recycling plant on the belt. The model will then be able to identify if it is a can or a bottle, etc... These are then further separated into products such as Coca-Cola, Sprite, etc... We will then tally these products into the database. The robotic arm will then pick up the materials and sort them into the appropriate bins for further processing.

Purpose and Vision (Background)

The world as we know now is driven by data. Companies that have a lot of data collected will be able to gain a competitive advantage over others that are unable to leverage this information. These data are able to aid companies in areas of but are not limited to:

- Make informed decisions to limit risks
- Be more efficient in the products that they produce
- Better understand customers
- Make better predictions of the market
- Etc...

Data collected through the means of humans is not impossible, but it would prove costly and less efficient. When we have a model that goes through the recycling plant, we can ensure that the data is unbiased and we could also easily perform data analysis given that we know the statistics of the model outputs.

Stakeholders

List people who have an interest in the project:

- Rahul Khanna (Project partner/ Founder)
 - Needs to be updated multiple times a week
 - Needs to be updated the progress, issues, and concerns of each team
 - Big decision maker, the project moves as he dictates
 - Everything that happens with this project is of concern to Rahul as he needs the information to ensure that the project is moving and proceeding as he paces and also aligned with his objectives
 - To make an informed decision, he needs all the updates of everyone on the team
- Engineering team
 - Needs to update internally daily/multiple times a week
 - Needs to be updated about the progress of other teams, or any changes in the objective of the project
 - Not decision-makers
- Seed investors
 - Updated monthly
 - Needs to be updated about the progress of the team
 - They make very few decisions but are very decisive
 - They will decide if they want to continue being an investor or talk to Rahul about objective changes
 - To make an informed decision, they need information about the progress and the objective of the team, they also need to experience the product itself

Preliminary Context

We know that there lies data that is worth it within recycled waste in recycling factories. By using cameras and artificial intelligence we can collect the data that we want and also sort items by category. We expect to be able to collect immense amounts of information that could prove essential to businesses.

Assumptions

With our cameras, we can capture good-quality images that will not negatively impact image recognition.

We can have items piled up on top of each other to the point where items are not recognizable.

We can use YOLO to train a model that is capable of identifying products and types of items on camera.

We can have a cloud that is available for us to deploy our model on.

The robotic arm will have the precision to pick up and drop items where needed.

We can finish the learning and deployment part of the project within the span of the capstone.

We can convince others that data like these are useful to the development of a company.

We are able to use 3D models to train an accurate detection from the image.

We are able to train YOLO to account for camera lens distortions.

The amount of products we are identifying does not scale too badly with the computing power needed to train the model.

Constraints

We need to make sure that the data we collect has high accuracy.

We need to be able to identify objects that are deformed.

We need proper lighting at night to be able to identify objects when even the lighting has changed.

We need objects to not be too scratched up beyond recognition

Dependencies

We are dependent on a 3D object model to start using that as a base for our model to perform recognition on video.

We are dependent on a model to finish to be able to deploy it onto the cloud

Producing the crushing of an object will require the 3D model to be made first

We require a cloud to be able to test deployment on the cloud

If the scale of our model is very large, we require proper computational power

Market Assessment and Competition Analysis

Strengths

The market is there and no one is doing an autonomous AI-trained robot sorting recycling that collects data

Weaknesses

Not all waste goes to a recycling center, and in America, many things are not recycled. We do not have a lot of funding and we are only creating 3D models of the object whenever a client requires this information, therefore there is most likely data that is not captured unless a company wants to, and we are unable to show them their data prior to them joining to convince them. All we could do is to use other companies as an example to prompt the project.

Opportunities

Many bottles/cans are not being dropped off at bottle drop and bottle drop does not accept a lot of different cans and bottles. By doing it this way, we are able to capture more data than what bottle drop could. Bottle drop is also biased in the sense that humans have to be the ones to choose and place the item into the machine, whereas all waste goes into a recycling plant.

Threats

There are bottle drops where people earn money dropping off bottles and we can easily use image recognition to identify the bottles/cans dropped. There is also an incentive to bottle drop and none with recycling.

There are also other alternatives to image recognition that exist. There are papers on the "classification of plastic bottle recycling based on image recognition". There are also easier ways to identify a product than to go through a whole process to rebuild what we are capturing in a 3D model.

Target Demographics (User Persona)

- Coca-Cola is a very well-known brand that sells a lot of drinks to other resellers, but they do not know how much is actually being consumed on a daily/monthly basis, but this project could show them insights
- Coca-Cola is unable to tell if people stopped drinking their product if resellers are always buying the same amount, but we will know if we look at the numbers in a recycling plant.
- Coca-Cola has just released a new advertisement, they want to have a method to tell how well their ad performed based on consumption. If the data is based on sales, we would not know the market trend instantly.
- Prime is a new drinks company, and although they are selling quickly, they do not know if they should increase production because of the increase in sales. Perhaps it is just the fact that they are new and everyone is buying them without consuming them first.

Requirements

We need to be able to create realistic 3D modeling of the object we are identifying, this includes different views and also deformation of the product. We are required to use YOLO to recognize a video of cans/bottles or any recyclable items moving on a belt by using a 3D model of the object. We are also required to deploy the model to the cloud to evaluate the video, providing real-time results.

User Stories and Features (Functional Requirements)

MoSCoW Prioritization:

- **Must Have:** the ability to identify recycling products with high accuracy
- **Should Have:** different ways a product can be squashed as produced by stable diffusion to be identified on the recycling belt
- **Could Have:** built 3D models of potential customers so that pitching the idea to them is much easier and more convincing
- **Will Not Have:** a robotic arm that will sort the items as needed

User Story	Feature	Priority	GitHub Issue	Dependencies
As a company, I want to know if my advertisement has led to high consumption or not so that I could increase production.	Product recognition	Must Have	TBD	N/A

As an environmentalist, I want to know the companies whose products are being recycled more or less so that we can get the company to promote recycling.	Product recognition	Must Have	TBD	N/A
As an environmentalist, I want to know the conditions of recycled items so that I can raise awareness of proper recycling where and when needed.	Product condition recognition	Could Have	TBD	N/A
As a company, I want to be able to tell the trends of consumers beforehand so that I can alter production lines.	Product recognition	Must Have	TBD	N/A
As an investor, I want to know if a company is getting more consumers or if are they simply selling more and saturating the market.	Product recognition	Must Have	TBD	N/A

Non-Functional Requirements

- The identification time should be less than a second to allow time for robotic arm to act
- The robotic arm should be able to sort without needing to stop the belt
- The system should not be able to be tempered by others to trick the market
- The system should be able to have an increase in products desired without a more than linear computational time
- The code should be well structured and documented
- The system should not make it hard to add new products to identify
- The system should not violate any relevant data protection regulations

Data Requirements

The only thing that needs to be stored is a row for each item detected with features of product name, product type, time of detection, accuracy, and also an image of the detected product.

Integration Requirements

The project has to be deployed and integrated with Azure ML-OPS on the AWS cloud. The video of the conveyor belt is sent to the cloud and detection is stored and then outputted from the cloud onto the robotic arm for proper action.

User Interaction and Design

There will be no user interface, we could query the database to return the required information. As the customer, the product is the data that is collected into the database so the interaction is through a database.

Milestones and Timeline

Further planning and research are needed with the team to forge a schedule and to-dos. The first few things to do are:

- to create a GitHub repository with everyone in it
- Get to know everyone's strengths and weaknesses
- Create a database with the right structure
- Get to know what is possible within the time frame of our project
- Get to know when is a proper time to meet with team members to discuss progression

Goals and Success MetricsOpen

Goal	Metric	Baseline	Target	Tracking Method
Produce first prototype of can	3-D object	Human recognizable 3-D object	Colored and shaped like a famous can drink(coke)	eye-ball
Produce recognition prototype	Recognition accuracy	85%	90%	Test set (video)

Questions

- What are the tasks that will be created?
- How will the automation team work before the 3-D modeling team finishes an object?
- Who are the potential stakeholders that we are collecting data on?
- How long will this project be estimated to take?

Out of Scope

- The ability to identify a completely flattened and crushed can be predicted by capturing the colors of strips that are still visible on the crushed can.

Software Development Process (SDP)

[Principles](#)

[Process](#)

[Roles](#)

[Tooling](#)

[Definition of Done \(DoD\)](#)

[Release Cycle](#)

[Environments](#)

Principles

- We are responsive with our asynchronous communication and answer within 24 hours
- We are going to check the discord server for any updates regarding work, changes in roles, etc...
- We acknowledge that roles are often associated with more work and not just delegation of work
- There will be tasks set such that there will not be a lack of tasks for the next 2 weeks
- We agree to contact the project partner and set a meeting with him weekly
- We are responsible to be prepared for any upcoming meetings
- We will acknowledge each and everyone's opinion with an open mind and follow up with questions that encourage discussion
- All changes or features are to be created on a branch and not on the main/master branch
- Merge requests must be reviewed by another member before merging
- We will use a kanban board or something similar to keep track of tasks
- A work task needs to be properly documented with technical descriptions, dependencies, and estimation before anyone can work on it
- We will not take more than 1 task at a time unless there are dependencies and a task has to be put on hold, making the person take another task while putting that on hold
- Tasks that are finished shall be seen as a pull request on GitHub and moved to the in-review section on a kanban board
- Tasks at most require a day of work to finish

Process

The process has yet to be decided, but assuming the structure and principles of our team. It is best to have a kanban board with a 2-week sprint. We will work together to put up a set of tasks that is laid out on a kanban board. These tasks have to adhere to the principles above. Every two weeks we will have a meeting to discuss the tasks for the next 2 weeks. This will allow us to

complete our project in a timely manner and also promote asynchronous flexible collaboration. At the end of the 2-week sprint, we are to ensure that all tasks are settled or pushed to the backlog to keep the project flowing.

Roles

Project management - This person is responsible for ensuring the project goes smoothly and is on track with goals.

Scrum master - This person will be in charge of running meetings to keep everyone on track and also managing the backlog and future tasks.

Data Engineer - This person is responsible for interpreting the data coming from visual outputs of the data into the input of the model, finding ways to reduce noise and improving accuracy

Product management - This person is in charge of making sure the project proceeds with the aligned objectives set out. This person is also in charge of notifying everyone if there is a change in objective.

Front end - This person is responsible for linking the backend data to the front end as well as the UI/UX of the project.

Back end - This person is responsible for data collection and making sure security vulnerabilities are addressed and is efficient when dealing with clients.

Deployment - This person is responsible for deploying the project on Azure with MLOPS, making sure that the deployment is smooth, and the training is asynchronous and effortless. This is also where a deployment should feedback to the team if there are any issues with the latest update.

Tooling

Version Control	GitHub / GitLab (unconfirmed)
Project Management	GitHub Issues and Projects, GitLab equivalent (unconfirmed)
Documentation	https://github.com/withastro/starlight (recommended), Astro, Sphinx, Read the Docs, Markdoc, Confluence, Notion, GitHub Wiki, README, etc.(unconfirmed)
Deployment	Kubernetes on Azure
Linting and Formatting	Prettier
ML technique	YOLO - You only look once with Azure MLOPS

IDE	Visual Studio Code
-----	--------------------

Definition of Done (DoD)

- All temporary programmer's testing code needs to be removed
- Code needs to be well commented or easily understandable
- Code has to be runnable and complete
- Code has to pass all test cases
- Code has to be free of linting issues
- Changes are made on a new branch and a pull request for main is made
- Any documentation is updated(if necessary)

Release Cycle

- Automatically check for linting and compilation errors on merge
- Automatically deployed to development on every merge to main
- Use semantic versioning to track models with MAJOR.minor.patch
 - Increment major for updates that are released to production and consist of major refactoring or big changes that fulfill objectives
 - Increment minor for new features
 - Increment patch for bug fixes

Environments

Environment	Infrastructure	Deployment	What is it for?	Monitoring
Production	AWS Azure MLOPS through CI/CD	Release	Clients view of the product, track data, etc..., for each recycling center	Prometheus, Grafana, Sentry
Staging (Test)	AWS Azure MLOPS through CI/CD	PR	New unreleased features and integration tests, used for demo	Sentry
Dev	Local (macOS and Windows)	PR	Development and unit tests	N/A

Software Design and Architecture

[Introduction](#)

[Architectural Goals and Principles](#)

[System Overview](#)

[Architectural Patterns](#)

[Component Descriptions](#)

[Data Management](#)

[Interface Design](#)

[Considerations](#)

[Security](#)

[Performance](#)

[Maintenance and Support](#)

[Deployment Strategy](#)

[Testing Strategy](#)

[Glossary](#)

Introduction

This document delineates the architecture of a project aimed at developing an advanced recycling AI system, designed to accurately identify and categorize various recyclable materials based on images. The system's ultimate objectives revolve around optimizing waste management processes, enhancing recycling efficiency, and contributing positively to environmental sustainability.

Architectural Goals and Principles

The architecture of the advanced recycling AI system is meticulously crafted to align with a set of specific goals and guiding principles, ensuring the project's success, efficiency, and adaptability to future needs and technologies.

Key Architectural Goals:

Accuracy and Reliability: Ensuring that the system accurately identifies and categorizes various types of recyclable materials based on the provided images. This is critical for optimizing waste management and recycling processes.

Scalability: The architecture must support the system's growth, handling an increasing amount of data and user interactions seamlessly. This includes the capability to scale both vertically and horizontally to accommodate growing computational and storage needs.

Real-Time Processing: Given the nature of waste sorting and recycling operations, the system must be capable of processing images and providing results in real-time, ensuring swift and efficient waste management.

Architectural Principles Guiding the Design:

Modularity: Designing the system in a modular fashion, breaking down the application into discrete, interchangeable components. This facilitates easier updates, maintenance, and scalability.

Separation of Concerns: Ensuring that different components of the system are responsible for specific functionalities, reducing complexity and enhancing the system's overall maintainability.

System Overview

Main Components and Their Interactions:

User Interface (UI):

Technology: React for frontend development, utilizing Redux for state management.

Functionality: Provides users with an intuitive and responsive interface to interact with the system. Users can upload images of recyclable materials, view results of material identification, and access historical data and analytics.

Interaction: Communicates with the Backend Server via RESTful APIs to send image data and receive results.

Backend Server:

Technology: Node.js with Express for server-side operations, ensuring a lightweight and efficient handling of requests.

Functionality: Processes image data received from the UI, manages user sessions, and interacts with the AI Model and Database to facilitate the material identification process.

Interaction: Communicates with the AI Model to send image data for processing and with the Database to retrieve and store information.

AI Model:

Technology: TensorFlow or PyTorch for building and running the deep learning model.

Functionality: Utilizes a trained deep learning model to analyze image data and identify recyclable materials. The model is trained on a diverse dataset of recyclable materials to ensure accuracy and reliability.

Interaction: Receives image data from the Backend Server, processes it, and sends back the identification results.

Database:

Technology: MongoDB for a scalable and flexible NoSQL data storage solution.

Functionality: Stores user data, historical analysis results, and any other necessary data for the system's operations.

Interaction: Interacts with the Backend Server to retrieve and store data as needed.

Third-Party Integrations:

Functionality: The system is designed to integrate seamlessly with various third-party services, such as waste management databases, recycling facility APIs, and additional analytics tools, to enhance its capabilities and provide more comprehensive services to the users.

Interaction: The Backend Server handles interactions with third-party services, sending and receiving data as needed to complement the system's internal operations.

Architectural Patterns

Microservices:

Usage:

Potential utilization for handling different functionalities within the Backend Server and AI Model components.

Rationale:

Adopting a microservices architecture can significantly enhance the scalability and maintainability of the system. Each microservice can be developed, deployed, and scaled independently, reducing the complexity of updates and ensuring that the system can handle increased load by scaling specific services as needed.

Architectural Decisions:

The decision to use or not use microservices would depend on the anticipated scale of the system and the complexity of its operations. A microservices architecture could introduce additional complexity in terms of service orchestration and network latency, and as such, the benefits need to be carefully weighed against these potential challenges.

Needs more research

Component Descriptions

User Interface (UI):

Responsibilities:

Handles all user interactions, including input and navigation.

Displays data and results from the AI model in an accessible and user-friendly manner.

Provides visualizations for the AI model's output, aiding in data interpretation.

Role in the System:

Acts as the main point of interaction for end-users, ensuring that they can effectively utilize the system's capabilities.

Translates complex AI results into understandable and actionable insights.

Technology: Implemented using React for its component-based architecture, allowing for a modular and maintainable UI.

Backend Server:

Responsibilities:

Processes requests from the UI, managing the flow of data to and from the AI Model and the Database.

Implements business logic and validation, ensuring data integrity and consistency.

Interfaces with third-party services and APIs, extending the system's functionality.

Role in the System:

Acts as the intermediary between the UI and the rest of the system, ensuring seamless data exchange and processing.

Enables the integration of additional services and APIs, enhancing the system's capabilities.

Technology: Built using Node.js and Express, providing a scalable and efficient server-side solution.

AI Model:

Responsibilities:

Analyzes input data, identifying materials and making recycling recommendations.

Learns from new data, continuously improving its accuracy and effectiveness.

Provides detailed output, including confidence scores and explanations of its recommendations.

Role in the System:

Delivers the core functionality of the system, enabling automated and intelligent recycling recommendations.

Ensures that the system adapts to new data, maintaining its relevance and accuracy over time.

Technology: Implemented using TensorFlow or PyTorch, depending on specific requirements and preferences.

Database:

Responsibilities:

Stores all data required by the system, including user inputs, AI model outputs, and additional resources.

Manages data access, ensuring data integrity and security.

Provides efficient query mechanisms, ensuring quick and reliable data retrieval.

Role in the System:

Acts as the central repository for all data, ensuring that the system has access to the necessary resources for optimal performance.

Supports the AI Model and Backend Server, providing a solid foundation for data storage and retrieval.

Technology: MongoDB is chosen for its scalability and flexibility, particularly in handling unstructured data.

Data Management

Unsure yet. Needs more research

Interface Design

Unsure yet. Needs more research

Considerations

Security

Unauthorized Access: Unauthorized users could gain access to sensitive user data or system functionalities.

Data Interception: Data transmitted over the network could be intercepted and exploited.

Injection Attacks: SQL Injection or other types of injection attacks could manipulate the system to reveal sensitive information or perform unauthorized actions.

Performance

High Availability: The system should be available 99.9% of the time, ensuring consistent access for users.

Scalability: The system should be able to handle an increasing number of users and data volume, scaling up resources as needed.

Low Latency: Response times for user interactions and data processing should be kept minimal, aiming for sub-second latencies.

Efficient Resource Utilization: The system should make optimal use of hardware and software resources to provide high performance.

Maintenance and Support

Dedicated Maintenance Team: A specialized team will be assigned to handle system maintenance, updates, and user support. This team will have in-depth knowledge of the system's architecture, technologies used, and the domain of advanced recycling.

Skills and Expertise: The team members should possess skills in the following areas:

Programming languages and frameworks used in the project (e.g., Python, JavaScript, React, Node.js).

Database management and optimization.

Cloud infrastructure and container orchestration (e.g., AWS, Kubernetes).

Security best practices.

Troubleshooting and problem-solving.

Training and Onboarding: New members joining the maintenance team will go through a comprehensive onboarding process to familiarize them with the system and its components.

Deployment Strategy

1. Environments:

Development: This is where active development occurs. Developers have full access to deploy and test their code. The environment is typically less robust than production.

Testing/QA: After development, the application is moved to a testing environment where QA teams evaluate it for bugs, performance issues, and other discrepancies. This environment mirrors the production setup to catch any environment-specific issues.

Staging: This is the pre-production phase where the application undergoes final tests. It's a replica of the production environment used for final checks and client reviews.

2. Configuration:

Automation: Scripts and templates (e.g., Terraform, CloudFormation) will be used for automating the deployment and configuration of resources across environments.

Monitoring and Logging: Tools like CloudWatch, ELK Stack, or Datadog will be set up for real-time monitoring, logging, and alerts.

Testing Strategy

1. Unit Testing:

Objective: To test individual units of code for correctness.

Tools: JUnit, NUnit, pytest, etc.

Practice: Developers will write unit tests alongside the code to test individual functions or methods.

Environment: Primarily conducted locally and in the development environment.

Frequency: Every time a change is made or a new feature is added.

2. Integration Testing:

Objective: To test the interaction between integrated units/modules to expose interface defects.

Tools: TestNG, Postman for API testing, etc.

Practice: After unit tests, components are combined and tested as a group.

Environment: Development and Testing/QA environments.

Frequency: Conducted after significant changes or additions to the system.

Glossary

N/A